1. In the 2020 US Presidential election, there was a significant gap between the probability of a Biden victory assigned by poll aggregators (e.g., fivethirtyeight.com) and betting markets. According to Bayesian decision theory, a rational agent should choose actions (in this case, bet size) by writing down:

   - a probability distribution $P(X)$ over the state X of the world describing what they think will happen,
   - a utility function U describing the value of each action under a certain state of the world,

   and then choose the action that maximizes their expected utility (equivalently, minimizes expected risk). That is, a rational agent will choose

$$a^* = \arg\max_a \mathbb{E}_{X\sim P}\left[U(a, X)\right].$$

   Suppose that Victor is a rational agent with d dollars of total wealth, and that he thinks P(Biden wins) $= 0.85$ (the fivethirtyeight prediction), and that betting markets give even money bets at P(Biden wins) $= 0.6$.

   (a) Prove that if Victor's utility function is just the amount of money he has then he'll bet d (his entire bankroll) on Biden winning. Does this kind of linear utility of money make sense?

   (b) Now, suppose that Victor's utility from y dollars is simply $U(y) = \log(y)$. How much (of his d dollars) should he bet on Biden winning?

   (c) Suppose you had a net worth of $\$500{,}000$. Would you personally make the bet from part (b)?

(a) Assume Victor will bet w dollars

So if Biden wins, Victor can have $w+d$ dollars

if Biden loses, victor can have $d-w$ dollars

$\mathbb{E}[U] = 0.85\times(d+w)+0.15\times(d-w) = d+0.7w$.

So therefore. $\mathbb{E}[U]$ is strictly increasing with w. and the upper bound of w is d.

And we'll bet d on Biden winning.

This kind of linear utility doesn't make sense. because;

① It will lead Victor to pay for infinite money for this bet.

② People tend to be risk-averse in real life compared to risk-neutral.

(b) $\mathbb{E}[U|y] = 0.85\cdot\log(d+w)+0.15\cdot\log(d-w)$

Pac. w.r.t w: $\frac{\partial\mathbb{E}[U|y]}{\partial w} = 0.85\cdot\frac{1}{d+w} - 0.15\cdot\frac{1}{d-w}$ (*)

Set (*) $= 0 \Rightarrow$ $0.85\cdot\frac{1}{d+w} = 0.15\cdot\frac{1}{d-w} \Rightarrow$ $w = 0.7d$

(c) Suppose I bet from the part (b):

If wins: $U = \log(\cdot)$

2. Suppose that Y is a categorical random variable, and X are covariates that we wish to use to predict Y. A loss function L on categorical data is called a *proper scoring rule* if the predictive function f^* that minimizes the implied risk is the conditional probability of Y given X. That is, if

$$f^* := \arg\min_f \mathbb{E}_{X,Y}\left[L\left(Y, f(X)\right)\right] \text{ is given by } f^*(x) = P(Y|X = x).$$

Prove that the Cross Entropy loss is a proper scoring rule in the case where Y is binary.

Cross Entropy loss $L(Y, f(X)) = -Y \cdot \log(f(X)) - (1-Y)\cdot\log(1-f(X))$ where $f(X) \in (0,1)$ is our predicted probability that $Y=1$

If Y is a binary variable, set $p = P(Y=1 \mid X=x)$. Thus, $1-p = P(Y=0 \mid X=x)$

$$\begin{aligned}
\mathbb{E}_{X,Y}[L(Y, f(X))] &= \mathbb{E}_{X,Y}\left[-Y\cdot\log(f(X)) - (1-Y)\cdot\log(1-f(X))\right] \\
&= \sum_x \sum_{y\in\{0,1\}} P(X=x, Y=y)\cdot\left(-y\log(f(x)) - (1-y)\log(1-f(x))\right) \\
&= \sum_x \left(P(X=x, Y=0)\cdot\left(-\log(1-f(x))\right) + P(X=x, Y=1)\cdot\left(-\log f(x)\right)\right) \\
&= \sum_x \left(P(Y=0 \mid X=x)\cdot P(X=x)\cdot(-\log(1-f(x))) + P(Y=1 \mid X=x)\cdot P(X=x)\cdot(-\log f(x))\right) \\
&= \sum_x \left(((1-p)\cdot P(X=x)\cdot(-\log(1-f(x))) + p\cdot P(X=x)\cdot(-\log(f(x)))\right) \quad (*)
\end{aligned}$$

Now we should minimize $(*)$ over $f(x)$

$$\frac{\partial(*)}{\partial f(x)} = \sum_x \left((1-p)\cdot P(X=x)\cdot\frac{1}{1-f(x)} + p\cdot P(X=x)\cdot\frac{1}{-f(x)}\right)$$

Since each x is independent here, we can separately treat them.

$$\forall x_i, \quad (1-p_i)\cdot P(X=x_i)\cdot\frac{1}{1-f(x_i)} + p_i\cdot P(X=x_i)\cdot\frac{1}{-f(x_i)} = 0$$

$$\Leftrightarrow \quad \forall x_i, \quad P(X=x_i)\cdot\left(\frac{1-p_i}{1-f(x_i)} - \frac{p_i}{f(x_i)}\right) = 0$$

$$\Rightarrow \quad \forall x_i, \quad P(X=x_i)\cdot\frac{f(x_i) - p_i f(x_i) - p_i + p_i f(x_i)}{(1-f(x_i))\,f(x_i)} = 0$$

$$\Leftarrow \quad \forall x_i, \quad P(X=x_i)\cdot\frac{f(x_i) - p_i}{(1-f(x_i))\,f(x_i)} = 0$$

$$\Leftarrow \quad \forall x_i, \quad f^*(x_i) = p_i = P(Y=1 \mid X=x_i)$$

$$\Leftarrow \quad f^*(x) = P(Y=1 \mid X=x)$$

Therefore, cross-Entropy loss is a proper scoring rule

Question 3

$$\sum_{t=1}^{\infty} \rho_t^2 < \infty \quad \text{requires convergence, implying} \quad \lim_{t \to \infty} \rho_t = 0.$$

This is theoretically set up for minimizing expected loss under the true data distribution:

$$\mathbb{E}_{x,y} \left[L(y, f(x)) \right],$$

implying an infinite amount of data points.

The decaying step is necessary here because SGD uses noisy gradient estimators, and it needs convergence to cancel the noise impacts.

In real machine-learning practice, we are minimizing the empirical learning objective:

$$\hat{L} = \sum_{i=1}^{n} L(y_i, f(x_i)),$$

and hence only finite data points are used.

With finite data, if we converge exactly to the minimum, it might cause overfitting. Choosing a constant ρ instead causes SGD to oscillate near the minimum, which acts as an implicit form of regularization.

Also, in real-life practice, we usually set up early-stopping criteria, and therefore a constant step could achieve approximately the same outcome as a decaying step can, so there is no point in setting a decaying step.

Question 4: Normality of Residuals

To check whether square error loss makes sense for the ALE dataset, I fit four models (Linear Regression, SVM, Decision Tree, and Neural Network) to predict ale from the first four variables. I split the data 80/20, standardized the features, and computed the residuals on the test set. For the Decision Tree I set max_depth= 5, because a default tree memorizes the training data and its residuals are all zero. I then checked the residuals for normality with Q-Q plots and the normaltest from scipy.

In Figure 1, most points are close to the line in the middle for all four models. The Linear Regression and SVM plots have a clear outlier at the top right, and the SVM also curves away from the line on the upper end. The Decision Tree and Neural Network plots look closer to the line overall, though the Decision Tree also has two points that sit above it at the top.

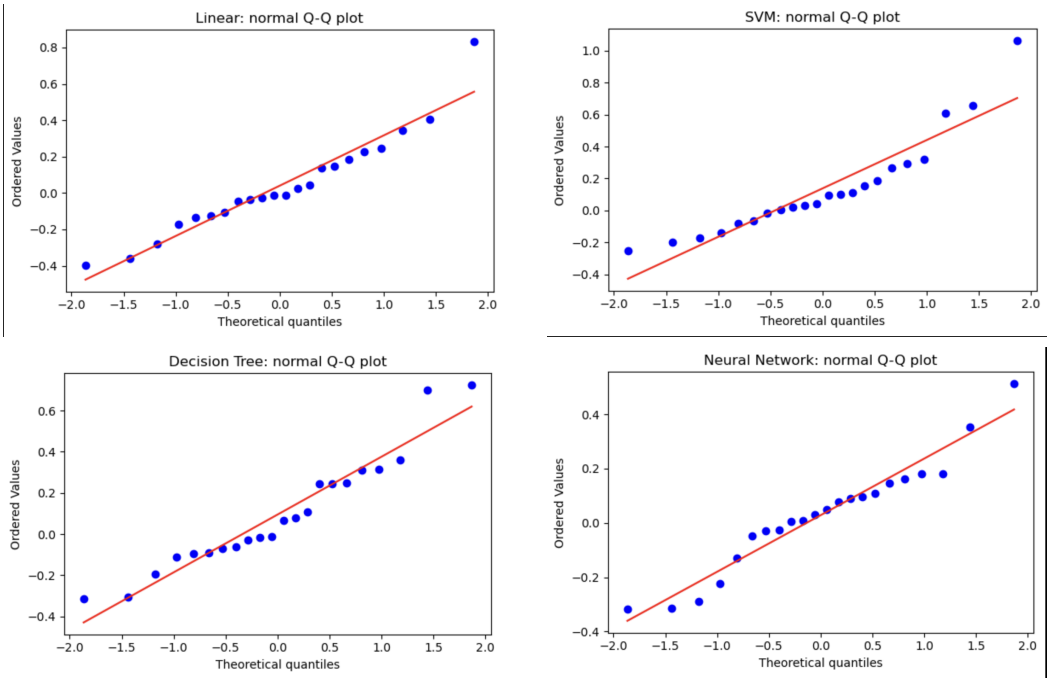


Figure 1: Q-Q plots of test residuals for different regression models. (a) Linear Regression, (b) SVM, (c) Decision Tree, (d) Neural Network.

The test statistics and p-values are in Table 1. Using a 0.05 cutoff, the Linear Regression ($p = 0.0260$) and SVM ($p = 0.0021$) residuals are not normal, so we **reject** normality for both. The Decision Tree ($p = 0.2038$) and Neural Network ($p = 0.6865$) are above 0.05, so we **fail to reject** normality for them. The Neural Network has the largest p-value and also the lowest test MAE (0.1534), so it is the model that best supports using square error loss.

One thing to keep in mind is that there are only 22 test points, so the test doesn't have much power. Failing to reject doesn't prove the residuals are normal, and the Linear Regression result in particular ($p = 0.026$) is only just below the cutoff. Overall, the normality assumption is only partly satisfied, depending on which model we use.

Table 1: Normality test results for test-set residuals.

Model	**Test MAE**	**Statistic**	p**-value**
Linear Regression	0.1954	7.2997	0.0260
SVM	0.2218	12.3384	0.0021
Decision Tree	0.2138	3.1817	0.2038
Neural Network	0.1534	0.7524	0.6865

Question 5: Automatic Differentiation

Classification (Breast Cancer Wisconsin Dataset)

The dataset has 569 observations, 30 features, and a binary label. I split it 80/20 into train (455) and test (114), and standardized the features with StandardScaler fit on the training set only. Scikit-learn's LogisticRegression (max_iter= 1000) got a test accuracy of **0.9825**.

Part (a): Custom Logistic Regression

I wrote logistic regression in PyTorch:

$$f(x_i) = \sigma(w^\top x_i + b), \qquad L(y_i, f(x_i)) = -\big[y_i \log f(x_i) + (1 - y_i) \log(1 - f(x_i))\big].$$

I clipped the predictions to $[10^{-7}, 1 - 10^{-7}]$ to avoid $\log(0)$, and trained for 5,000 epochs with torch.optim.SGD (lr= 0.01). The training loss went from 0.6931 to 0.0659 by epoch 4500. The test accuracy was **0.9737**, about 0.9% below sklearn. The gap is small and is probably because SGD is less exact than sklearn's solver.

Part (b): Probit Regression

Probit uses the standard normal CDF Φ instead of the sigmoid:

$$f(x_i) = \Phi(w^\top x_i + b).$$

Everything else was the same. The training loss went from 0.6931 to 0.0554, slightly lower than the sigmoid model, and the test accuracy was **0.9737**, the same as logistic regression.

Table 2: Test accuracy comparison for classification models.

Model	**Link Function**	**Test Accuracy**
Sklearn Logistic Regression	$\sigma(\cdot)$	0.9825
Custom Logistic Regression	$\sigma(\cdot)$	0.9737
Custom Probit Regression	$\Phi(\cdot)$	0.9737

Both custom models are close to sklearn, so the PyTorch code works. Logistic and probit got the same accuracy, so the link function doesn't matter much here.

Regression (ALE Dataset)

The ALE dataset has 107 observations. I used the first four variables (anchor_ratio, trans_range, node_density, iterations) to predict ale, with an 80/20 split (85 train, 22 test) and standardized features. Scikit-learn's LinearRegression got a test $R^2 = 0.6292$ and an MAE of **0.1954**.

Part (a): Squared Error Loss

I wrote linear regression $f(x_i) = w^\top x_i + b$ in PyTorch with squared error loss $L(y_i, f(x_i)) = (y_i - f(x_i))^2$. I trained it for 1,000 epochs with my own StochasticGradientDescent class (lr= 0.01). The training loss went from 1.0661 to 0.0506 by epoch 100 and then stayed flat. The test MAE was **0.1954**, exactly the same as sklearn, so my SGD works.

Part (b): Student-t Loss

Since the residuals in Question 4 were only partly normal, I also tried the negative log-likelihood of a Student-t distribution with $\nu = 5$, which has heavier tails:

$$L(y_i, f(x_i)) = -\log p(y_i - f(x_i),\, \nu = 5),$$

using torch.distributions.StudentT(df=5) with the same settings. The training loss went from 1.5161 to 0.9977, and the test MAE was **0.1941**, slightly better than the other two.

Table 3: Test MAE comparison for regression models.

Model	**Loss Function**	**Test MAE**
Sklearn Linear Regression	—	0.1954
Custom Linear Regression (SGD)	Squared Error	0.1954
Custom Linear Regression (Student-t)	t-distribution ($\nu = 5$)	0.1941

The improvement is small, but it fits with Question 4: the normality assumption behind squared error loss isn't fully met, and a heavier-tailed loss is less affected by outliers.